AILT9019 · AI Literacy II

Revised Project Proposal

Due: Saturday, 10 Oct (Week 6) · One submission per team, via the Moodle form

1. Background

Who is this for, what is stuck, why it matters this semester, and who or what it is not for. Also state where the idea comes from: which topic-list idea(s) and how far they were adapted, an external idea, or an original idea. A figure is optional.

Project title: Character Relationship Network for English Long Texts.

One sentence to describe:

Conclude the text input into a graph which assists readers to understand the relationships between the characters in the text.

Target users are undergraduate students in an English literature course who are reading a character-dense novel and need to keep track of who is who as they read chapter by chapter. The main pain point is that books such as Pride and Prejudice, Jane Eyre, or War and Peace contain many characters, name variants, titles, and indirect references, so readers constantly lose track of who is who — for example, whether "Lizzy" and "Elizabeth" are the same person — and have to flip back or re-read to recall how characters relate, which slows down reading and preparation for class discussion.

This matters for this semester because the project applies AI literacy and natural language processing in a practical reading-support scenario: it turns an English long text into a visual relationship graph that non-technical users can inspect. The idea is adapted from the course direction of text analysis and information extraction, narrowed to one feasible product: a character co-occurrence network for a single uploaded English book.

This project is not intended to replace literary criticism, produce plot summaries, classify exact relationship types, or make authoritative claims about the story. In the first version, an edge means two names often appear near each other in the text; it does not prove they are family, enemies, or romantic partners.

2. Project pipeline

End-to-end path: request → method → result, and what the team will actually build. A pipeline diagram is encouraged.

End-to-end path: the user uploads or pastes an English book text, the system identifies person names, applies a small v1 name-cleaning step, counts how often character pairs appear in the same paragraph or sliding window, and then generates an interactive weighted network graph.

The team will build a small local web app, most likely using Streamlit or Gradio with a Python backend. Since the first version only accepts English text, spaCy NER can be used to detect PERSON entities. This keeps the scope smaller and avoids the extra uncertainty of supporting multiple languages. NetworkX will build the graph, and PyVis or Plotly will display it in the browser.

For name variants, v1 will use a deliberately limited approach: a small built-in alias mapping table for the selected test book, plus simple title-word handling for forms such as Mr., Mrs., Miss, and Dr. For example, "Elizabeth", "Lizzy", and "Miss Bennet" can be mapped through a hand-written alias table during testing. Fully automatic variant clustering is not in the v1 scope because it is error-prone and could merge different characters incorrectly.

The planned interface is simple: upload an English txt file or paste English text, optionally provide a character name list or alias table, adjust parameters such as window size and minimum co-occurrence count, then click generate. The output will be an interactive graph rendered in the browser (standalone HTML), where users can zoom, drag nodes, hover to see co-occurrence counts, and filter weak edges; optionally accompanied by a downloadable node/edge table (CSV). The exact output format depends on the outcome of the AI/NLP processing (e.g. which names spaCy detects and how co-occurrence is counted), and will be finalised and stated explicitly in the final proposal. The main page will include a short "How to use" panel — upload or paste English text, optionally adjust parameters, then click Generate — so that first-time users see the instructions inside the app itself, not only in the README.

A possible pipeline diagram is: Upload English text -> English person-name recognition -> Small alias/title cleanup -> Co-occurrence counting -> Network graph generation -> Interactive display.

3. Evaluation and test

How the team will tell the product works, including at least one case it should fail safely or refuse.

The tool will be tested on 2-3 English texts with publicly known character structures, such as Pride and Prejudice, Jane Eyre, or selected chapters from an English public-domain novel. A small manual reference list of main characters and expected major relationships will be prepared as a ground truth for comparison.

The first pass criteria are modest: the top 10 main characters by frequency should mostly match the manual list, with around 70% recall as a target; major relationships should appear as visible graph edges; and the central characters should appear as high-degree or visually central nodes. For example, Elizabeth and Darcy should have a clear edge in Pride and Prejudice.

Failure testing will include an empty file, a non-text file, a non-English text, and an English text with few or no character relationships, such as a theory-style article. In these cases, the app should show a clear message such as "please upload an English plain-text file" or "not enough character relationships detected" instead of crashing or forcing a meaningless graph.

4. Distribution and accessibility

How someone else can open the app: API setup, first use without a coding background, and (if in scope) other agent environments.

The first version will be distributed as a GitHub repository with README.md, requirements.txt, and the app source code. The README will provide a step-by-step setup (install Python → create a virtual environment → pip install -r requirements.txt → streamlit run app.py), and before the final submission we will have one non-coding classmate follow the walkthrough from a fresh machine and record the result as evidence.

The final display format will be a local browser-based web app, not a native desktop or mobile app. The graph itself can also be exported as a standalone HTML file or image if time allows. No user account system or cloud deployment is planned for the first version.

The core version does not need an external API because name recognition, co-occurrence counting, and visualization can run locally with open-source Python packages. If an optional online API or LLM is later added to classify relationship types, the README will explain API-key setup, but that will be treated as a should-have extension rather than the main dependency.

5. Ethics and risks

An explicit “does not”, plus main misuse or failure risks and how they are mitigated.

Explicit does not: the tool does not determine the real nature of a relationship, does not generate character biographies or plot summaries, does not redistribute the uploaded text, does not upload user text to a remote service in the core version, and does not replace close reading or academic judgment. It also does not attempt fully automatic name-variant clustering in v1. The tool also does not include any copyright-detection feature for uploaded texts; it simply processes what the user provides and keeps everything local.

Main risk 1 is name recognition error, such as missing an important character or mistaking a place for a person. The mitigation is to display the recognized name list and, if time allows, let users manually add or remove names.

Main risk 2 is name normalization error, such as merging two different characters or splitting one character into several aliases. The v1 mitigation is intentionally conservative: use only a small alias table and title-word rules, and avoid automatic clustering unless there is time to test it carefully.

Main risk 3 is over-interpreting co-occurrence as a real social relationship. The interface and README should clearly state that edge weight means co-occurrence frequency only.

Main risk 4 is privacy: users may upload sensitive or personal text. The first version runs fully locally and does not store or share the uploaded text.

6. Checkpoints

Must-have and should-have acceptance items, plus a timeline to the Week 4 draft. A timeline figure is encouraged.

Must-have items: read English txt input; identify English person names using spaCy or a similar local NLP package; handle a small built-in alias mapping table and basic title words such as Mr., Mrs., Miss, and Dr.; count character co-occurrences by paragraph or sliding window; generate a NetworkX graph; display an interactive graph in Streamlit or Gradio; make edge thickness reflect co-occurrence count; and test on at least one English book.

Should-have items: support manual add/remove names; allow users to upload or edit the alias table; improve simple name normalization for variants such as "Elizabeth", "Lizzy", and "Miss Bennet"; allow users to adjust window size and minimum edge weight; export the graph as HTML or PNG; and optionally paste English text directly instead of only uploading a file. Fully automatic variant clustering is a later extension, not a v1 requirement.

Timeline:

Week 4 (25 Sep): submit the proposal draft; decide the first test book and its small alias list; run the spaCy NER spike.

Weeks 5–6: build the minimum version end to end — upload txt → recognise names → apply alias/title cleanup → count co-occurrence → display graph; submit the revised proposal (10 Oct).

Weeks 7–8: test on one or two English books against the manual reference list, record failure cases, and reach the mid-term check standard (20 Oct).

Weeks 9–10: implement should-have items — editable alias table, export to HTML/PNG, paste-text input — and evaluate on more books.

Weeks 11–12: polish the interface, README, project webpage, and demo materials for the final deliverables (20 Nov).

7. Response to reviewers

For every review your proposal received: cite the Reviewer ID, name the suggestion in a few words, state adopted / partially adopted / not adopted, and give one line why. No penalty for declining — but every review gets a visible response. Three rows are usually enough.

Reviewer ID	Suggestion (short)	Response	Why (one line)
3036601238	Remove function: identifying copyright of the text/book	adopted	Identifying copyright is too complex to achieve.
3036601238	Put instructions for users in demo	adopted	Improve the experience of users.
3036601238	The output will be an interactive graph rendered in the browser (standalone HTML)	adopted	Graph is more straight forward.

Add rows as needed — one row p